

# Homework 7

Gabriela Rubio

April 16, 2026

1. (a) **Algorithm Description:** We can solve this algorithm by modifying Dijkstra's algorithm. Instead of tracking the total path weight we can track the bottleneck's weight. So, instead of keeping `dist[v]` as the minimum sum of edge weights from  $s$  to  $v$ , we can keep `bottleneck[v]` as the minimum possible maximum edge weight on any path from  $s$  to  $v$ . We start by initializing `bottleneck[s] =  $-\infty$`  (since a path with zero edges has no maximum), and `bottleneck[v] =  $\infty$`  for all other vertices. We use a min-heap priority queue keyed on bottleneck values. When we 'relax' an edge ( $u \rightarrow v$ ) we compute  $\max(\text{bottleneck}[u], w(u, v))$  instead of adding. If it is less than `bottleneck[v]` we update it and set `pred[v] = u`. Once a vertex is extracted from the heap its bottleneck value is already final.

(b) **Pseudocode:**

```
Algorithm MinBottleneckPaths(G, s)
Input: Directed graph G with adjacency lists and edge weights, source s
Output: Predecessor array pred[] for minimum bottleneck-weight path tree

for each vertex v in G:
    bottleneck[v] = infinity
    pred[v] = null
bottleneck[s] = -infinity
H = new MinHeap
H.insert(s, -infinity)

while H is not empty:
    u = H.extractMin()
    for each edge (u, v) with weight w(u,v) in adjacencyList[u]:
        b = max(bottleneck[u], w(u,v))
        if b < bottleneck[v]:
            bottleneck[v] = b
            pred[v] = u
            H.decreaseKey(v, b)

return pred[]
```

- (c) **Correctness:** The algorithm is correct because when the vertex  $u$  is extracted from the heap, `bottleneck[u]` is already minimal. Any other path to  $u$  that goes through some other vertex  $x$  still in the heap has `bottleneck[x]  $\geq$  bottleneck[u]`, and since  $\max(\text{bottleneck}[x], \dots) \geq \text{bottleneck}[x]$ , no such path can improve `bottleneck[u]`. Then, when we relax edge ( $u \rightarrow v$ ) by computing  $\max(\text{bottleneck}[u], w(u, v))$  we correctly compute the bottleneck of the best path to  $v$  that uses  $u$  as its second to last vertex. The predecessor array records the last edge of the best path to each vertex, which then forms the minimum bottleneck-weight path tree.
  - (d) **Time Analysis:** Initialization takes  $O(n)$ . The main loop extracts each vertex at most once from the heap which takes  $O(\log n)$  per extraction, and then each edge triggers at most one `decreaseKey` which also takes  $O(\log n)$ . Since there are  $n$  extractions and  $m$  edge relaxations the total time is  $O((n + m) \log n)$ .
2. The algorithm is not correct because adding a constant  $-W$  to every edge will change the path weights by different amounts depending on how many edges are in the given path. A path with  $k$  edges will get its weight increased by  $k \cdot (-W)$ , so paths with more edges are penalized more, and this can change which path is the shortest.

Counterexample: Given a graph  $G$  with vertices  $s, a, t$  with edges:  $s \rightarrow a$  with weight 0,  $a \rightarrow t$  with weight  $-3$ , and  $s \rightarrow t$  with weight  $-2$ . In  $G$ , the path  $s \rightarrow a \rightarrow t$  has total weight  $-3$  and  $s \rightarrow t$  has weight  $-2$ , so the shortest path is  $s \rightarrow a \rightarrow t$ .

The smallest edge weight is  $W = -3$ , so we add  $-W = 3$  to every edge. In  $G'$ :  $s \rightarrow a$  has weight 3,  $a \rightarrow t$  has weight 0,  $s \rightarrow t$  has weight 1. Now  $s \rightarrow a \rightarrow t$  has total weight 3 and  $s \rightarrow t$  has weight 1, so Dijkstra's picks  $s \rightarrow t$  in  $G'$ . But the correct shortest path in  $G$  is  $s \rightarrow a \rightarrow t$ , so the algorithm ends up giving the wrong answer.

3. (a) **Algorithm Description:** We can solve this by reducing it to a minimum spanning tree problem. We start by assigning weight 0 to every blue edge and weight 1 to every red edge, then the total weight of any spanning tree equals exactly the number of red edges it contains, so finding an MST under these weights gives us a spanning tree with the fewest red edges. We can then use the Kruskal's algorithm we learned and sort all edges with blue edges before red edges, then add the edges in that order using a union-find structure and skip any edge that would form a cycle.

- (b) **Pseudocode:**

```

Algorithm MinRedSpanningTree(G)
Input: Undirected connected graph G with adjacency lists,
       edges colored blue or red
Output: Spanning tree T with minimum number of red edges

Sort edges making sure all blue edges come before red edges
Initialize union-find on vertices of G
T = empty set

for each edge (u, v) in sorted order:
    if Find(u) != Find(v):
        Union(u, v)
        T.add(u, v)

return T

```

- (c) **Correctness:** The algorithm is correct because it is exactly Kruskal's MST algorithm with edge weights 0 for blue and 1 for red, and Kruskal's produces a minimum spanning tree. Since the total weight of a spanning tree under these weights is exactly its number of red edges, the MST minimizes the number of red edges. So the algorithm never includes a red edge when a blue edge could connect the same two components.
- (d) **Time Analysis:** Sorting the edges takes  $O(m \log m) = O(m \log n)$ . The union-find operations over all edges take  $O(m\alpha(n))$  which is effectively  $O(m)$ . The final check over all vertices takes  $O(n)$ . The total time is  $O((n + m) \log n)$ .